

MIPSolvers 使用教程

优化求解器：理论、接口与电力系统应用

内部技术教程

MIPSolvers v0.1

2026 年 5 月 8 日

Python 3.8 + pybind11 + HiGHS/SCIP

目录

- 1 优化求解器概论
- 2 Engine 数学格式
- 3 Python 接口使用
- 4 SCUC 问题建模与求解
- 5 工作进展与计划

什么是数学优化？

数学优化 (Mathematical Programming)

在满足约束条件的前提下，寻找使目标函数最小（或最大）的决策变量取值：

$$\min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad \text{s.t.} \quad g_i(\mathbf{x}) \leq 0, \quad h_j(\mathbf{x}) = 0$$

三要素：

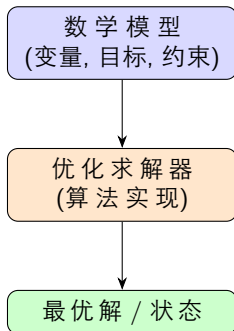
- 决策变量 $\mathbf{x}$ ：可调节量
- 目标函数 $f(\mathbf{x})$ ：优化指标
- 约束条件 g, h ：物理/逻辑限制

常见问题类型

LP	线性规划
QP	二次规划
NLP	非线性规划
MILP	混合整数线性规划
MINLP	混合整数非线性
LE	线性方程组
NLE	非线性方程组

求解器的作用与分类

求解器封装了数值优化算法，提供统一接口：



求解器	类型	许可
Gurobi	LP/MILP	商业
CPLEX	LP/MILP	商业
HiGHS	LP/MILP	开源
SCIP	MILP	开源
Ipopt	NLP	开源
GLPK	LP/MILP	开源

MIPSolvers 统一以上接口

核心算法：

- 单纯形法 / 对偶单纯形法 (LP)
- 内点法 (LP, NLP)
- 分支定界法 (MILP)
- 切平面法、列生成法

主要应用场景

通用领域：

- 供应链与物流优化（运输规划、选址）
- 金融投资组合优化
- 生产排程与资源分配
- 机器学习（正则化、SVM）
- 通信网络路由

电力系统专项：

- 机组组合（UC/SCUC）：确定每台机组的开/停计划
- 最优潮流（OPF）：最小化发电成本，满足电网约束
- 电力市场出清：计算边际电价（LMP）
- 输电规划：确定新建线路方案
- 储能调度：管理电池充放电计划

电力系统调度层级

年度/季度	检修计划规划
周	水电协调优化
日前	SCUC（机组组合）
小时前	SCED（经济调度）
实时	AGC 自动发电控制
毫秒级	保护与稳定控制

MIPSolvers 重点：日前 SCUC 及 SCED 的快速求解与建模

电力系统优化的理论难题（一）

1. 计算复杂性（NP 难）

SCUC 中有 G 台机组、 T 个时段：

$$\text{组合数} = 2^{G \times T}$$

- $G = 50, T = 24$ ：超过 10^{360} 种方案！
- 启停约束（最小开/停机时间）引入时间耦合
- 分段报价将连续变量线性化，增加辅助二值变量

2. 时间耦合约束

- 爬坡约束： $|p_t - p_{t-1}| \leq R_g$
- 最小开机时间： $\sum_{\tau=t}^{t+T^{up}-1} u_{\tau} \geq T^{up} \cdot (u_t - u_{t-1})$
- 使 LP 松弛的质量决定 B&B 效率

实际规模（华东电网示例）

机组数	~1000
节点数	~2000
线路数	~3000
时段数	24-96
二值变量	> 24000
求解时限	≤ 30 min

常用应对策略：

- Lagrangian 松弛
- Benders 分解
- 强化形式（Tight formulations）
- 列生成 / 分支定价
- 热启动（Warm start）

电力系统优化的理论难题（二）

3. 非凸性 (AC-OPF)

交流最优潮流引入非线性等式约束：

$$P_i = \sum_j |V_i| |V_j| (G_{ij} \cos \theta_{ij} + B_{ij} \sin \theta_{ij})$$

- 电压模/相角乘积为非凸
- 局部最优解众多，全局优化困难
- SDP / 多项式优化松弛方法

4. 不确定性 (随机优化)

- 风电/光伏出力不确定
- 负荷预测误差
- 随机 UC、鲁棒 UC、机会约束

5. 数值挑战

- Big-M 取值不当 → 数值病态
- 线路潮流约束引入大系数差异
- 稀疏 vs. 密集矩阵混合

MIPSolvers 目标

提供统一的求解器接口，使研究人员专注于：

- 建模与算法设计
- 快速原型验证
- 多求解器对比

而不是底层求解器调用的细节

支持的标准数学格式总览

类型	目标	变量域	可用求解器
LP	线性	连续	HiGHS, Gurobi, NativeLCQP, NativePDLP, NativePMLP
QP	二次	连续	HiGHS, Gurobi, NativeLCQP
NLP	非线性	连续	NativePMLP (内点法)
MILP	线性	混合整数	HiGHS, Gurobi, SCIP, NativeBranchAndCut
MINLP	非线性	混合整数	NativeBranchAndCut
LE	—	连续	NativeDirect (稀疏 LU 分解)
NLE	—	连续	NativeNewton (Newton-Raphson)

统一入口: `mipsolvers.engine.solve_lp / solve_milp / solve_nlp / ...`

自动选择: `solver="Auto"` 按优先级选取已安装且最适合的求解器

LP — 线性规划

标准形式:

$$\begin{aligned} & \min_{\mathbf{x}} \mathbf{c}^T \mathbf{x} \\ \text{s.t.} \quad & \begin{cases} \mathbf{A} \mathbf{x} \leq \mathbf{b} \\ \mathbf{A}_{eq} \mathbf{x} = \mathbf{b}_{eq} \\ \mathbf{l}_b \leq \mathbf{x} \leq \mathbf{u}_b \end{cases} \end{aligned}$$

Python 接口参数:

- $\mathbf{c}$: 目标系数向量 $\mathbb{R}^n$
- $\mathbf{A}$, $\mathbf{b}$: 不等式约束
- $\mathbf{A}_{eq}$, $\mathbf{b}_{eq}$: 等式约束 (可省略)
- $\mathbf{l}_b$, $\mathbf{u}_b$: 变量界 (可省略)
- `solver`: 求解器名称
- `maximize`: 默认 `False`

返回字典 `res`:

- `res['success']`: 是否求解成功
- `res['objective']`: 最优目标值
- `res['x']`: 最优解向量
- `res['status']`: 状态描述
- `res['duals']`: 对偶变量 (影子价格)
- `res['solver']`: 实际使用求解器
- `res['runtime_sec']`: 耗时

应用示例

生产计划、运输、网络流、电力经济调度

MILP — 混合整数线性规划

标准形式:

$$\min_{\mathbf{x}} \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{Ax} \leq \mathbf{b}, \quad l_b \leq \mathbf{x} \leq u_b$$

$$x_i \in \mathbb{Z} \text{ (或 } \{0, 1\}) \quad \forall i \in \mathcal{I}$$

额外参数:

- `vartypes`: 列表, 每元素为 "C" (连续)、"B" (0/1)、"I" (整数)
- `mip_gap`: MIP 容忍间隙 (默认 10^{-4})
- `time_limit_sec`: 时间上限

额外返回字段:

- `res['mip_gap']`: 实际间隙

算法: **Branch & Bound**

- 1 求解 LP 松弛, 得到下界
- 2 选择分数变量进行分支
- 3 剪枝: 界 + LP 松弛判断
- 4 更新上界 (可行整数解)

应用示例

背包/排程、设施选址、
机组组合、供应链规划

QP、NLP 与方程组求解

QP —二次规划:

$$\min \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{A} \mathbf{x} \leq \mathbf{b}$$

- $\mathbf{Q} \succeq 0$ (正半定) $\rightarrow$ 凸问题
- 参数: $\mathbf{Q}$ (稀疏/密集), 其余同 LP
- 应用: 投资组合、MPC 轨迹优化

NLP —非线性规划:

- 用户提供 $f, \nabla f, H_f, g, \nabla g$ 回调函数
- 内点法 (NativeIPMLP)
- 应用: AC-OPF、非线性工程优化

LE —线性方程组:

$$\mathbf{A} \mathbf{x} = \mathbf{b}, \quad \mathbf{A} \in \mathbb{R}^{m \times n}$$

- 稀疏 LU 分解 (NativeDirect)
- 适用: 潮流方程线性化、预条件

NLE —非线性方程组:

$$\mathbf{F}(\mathbf{x}) = \mathbf{0}, \quad \mathbf{F}: \mathbb{R}^n \rightarrow \mathbb{R}^n$$

- Newton-Raphson 方法
- 需提供 $\mathbf{F}$ 和 $\mathbf{J}_F$
- 适用: 潮流计算、稳态方程

两种 Python 接口

底层 Engine 接口

- 直接传 NumPy 矩阵/向量
- 适合矩阵结构已知的小规模问题
- 最低开销，最直接控制
- 返回字典 (res['x'], res['objective'])

```
mipsolvers.engine.solve_lp(...)  
mipsolvers.engine.solve_milp(...)
```

AML 代数建模接口

- 声明式建模：集合、参数、变量、约束
- 支持索引变量（多维下标）
- 可读性强，接近数学符号
- 自动管理矩阵构造
- 支持 LP / MILP / QP / NLP

```
mipsolvers.aml.Model(...)
```

导入模块：

```
1 import sys  
2 sys.path.insert(0,  
3     '/path/to/MIPSolvers/build_py')  
4 import mipsolvers  
5 aml = mipsolvers.aml  
6
```

底层接口：求解 LP（生产计划）

问题： $\max 5x_1 + 4x_2$

$$\text{s.t. } \begin{pmatrix} 6 & 4 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \leq \begin{pmatrix} 24 \\ 6 \end{pmatrix}, x_1, x_2 \geq 0$$

```
1 import numpy as np
2 import mipsolvers
3
4 c = np.array([-5.0, -4.0]) # min
5 A = np.array([[6.0, 4.0], [1.0, 2.0]])
6 b = np.array([24.0, 6.0])
7 lb = np.zeros(2)
8
9 res = mipsolvers.engine.solve_lp(
10     c=c, A=A, b=b, lb=lb,
11     solver="Auto")
12
```

```
1 print(res['status'])
2 # Optimal
3
4 print(-res['objective'])
5 # 21.0
6
7 print(res['x'])
8 # [3.0 1.5]
9
10 print(res['duals'])
11 # [0.75 0.5] (shadow prices)
12
```

完整示例： demo_01_lp_direct.py

查询可用求解器

```
mipsolvers.engine.list_solvers("LP")
```

底层接口：求解 MILP (0/1 背包)

问题：选物品使总价值最大，总重量 $\leq W$

$$\max \mathbf{v}^\top \mathbf{x}, \quad \mathbf{w}^\top \mathbf{x} \leq W, \quad x_i \in \{0, 1\}$$

```
1 weights = np.array([3,4,5,2,6,1],
2                       dtype=float)
3 values  = np.array([9,8,10,5,12,3],
4                       dtype=float)
5 W, n = 10.0, 6
6
7 res = mipsolvers.engine.solve_milp(
8     c = -values,
9     A = weights.reshape(1, n),
10    b = np.array([W]),
11    lb = np.zeros(n),
12    ub = np.ones(n),
13    vartypes = ["B"] * n,
14    solver = "Auto")
15
```

```
1 print(-res['objective'])
2 # 22.0
3
4 sel = [i for i in range(n)
5         if res['x'][i] > 0.5]
6 # 选了哪些物品
7
8 print(res['mip_gap'])
9 # 0.0 (精确最优)
10
11 print(res['solver'])
12 # HiGHS
13
```

完整示例： demo_02_milp_knapsack.py

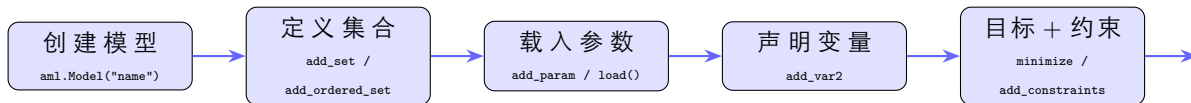
变量类型

"B" = 0/1 整型

"I" = 普通整型

"C" = 连续型

AML 代数建模工作流程



term
var_val

AML: 集合、参数与变量

集合 (索引域):

```
1 m = aml.Model("transport")
2
3 # 普通集合
4 I = m.add_set("I", ["A", "B", "C"])
5
6 # 有序集合 (支持 prev/next)
7 T = m.add_ordered_set("T",
8     ["t1", "t2", "t3", "t4"])
9
```

参数:

```
1 c = m.add_param("cost", dim=2)
2 c.load({
3     ("A", "t1"): 2.5,
4     ("A", "t2"): 3.0,
5     ("B", "t1"): 4.0,
6     ...
7 })
8 val = c[("A", "t1")] # => 2.5
9
```

决策变量:

```
1 # 1维变量 (indexed by I)
2 x1 = m.add_var("x", I,
3     aml.VarType.Continuous,
4     lb=0.0)
5
6 # 2维变量 (indexed by I x T)
7 x2 = m.add_var2("y", I, T,
8     aml.VarType.Binary)
9
10 # 访问方式
11 xv = x2[("A", "t1")] # VarRef
12
```

变量类型:

- VarType.Continuous
- VarType.Binary
- VarType.Integer

AML: 目标函数与约束

目标函数 (使用 `sum_over`):

```
1 # min sum_{i,t} cost[i,t] * x[i,t]
2 m.minimize(
3     aml.sum_over(I, lambda i:
4         aml.sum_over(T, lambda t:
5             cost[(i,t)] * x2[(i,t)])))
6
```

批量约束 `add_constraints`:

```
1 # 对集合I中每个i添加约束
2 m.add_constraints("cap", I,
3     lambda i:
4         aml.sum_over(T, lambda t:
5             x2[(i,t)] * 1.0) <= cap[i])
6
```

单个约束 `add_constraint`:

```
1 # 线性表达式比较
2 lhs = 1.0*x1[i] - 50.0*x2[i]
3 m.add_constraint(
4     lhs >= 0.0, "pmin_i")
5
6 # 表达式 == 常数
7 m.add_constraint(
8     1.0*a + 1.0*b == demand,
9     "balance")
10
```

表达式构造规则

```
float * VarRef → LinearExpr
LinearExpr ± LinearExpr → LinearExpr
LinearExpr ≥ float → TempConstr
```

AML: 求解配置与结果提取

求解选项:

```
1 opts = aml.SolveOptions()
2 opts.solver_name = ""      # Auto
3 opts.verbosity = 0        # 0=silent
4 opts.mip_gap_tol = 1e-3    # MIP gap
5 opts.time_limit = 300.0    # seconds
6 opts.max_iter = 10000
7
8 result = m.solve(opts)
9
```

基本结果:

```
1 print(result.termination_status)
2 # TerminationStatus.Optimal
3
4 print(result.objective_value)
5 # 320.5
6
7 print(result.solver_used)
8 # HiGHS
9
```

提取变量值:

```
1 # 单个变量
2 v = result.var_value(x2[("A", "t1")])
3
4 # 整个变量数组 -> dict
5 vals = result.array_values(x2)
6 # {"A", "t1": 2.5,
7 #   ("A", "t2"): 0.0, ...}
8 for key, val in vals.items():
9     if val > 0.01:
10         print(key, val)
11
```

提取对偶变量:

```
1 # 批量约束的对偶值
2 ca = m.add_constraints(
3     "c", I, lambda i: ...
4
5 duals = result.dual_array(ca)
6 # {"A": -0.75, "B": -0.50}
```

AML 示例：运输问题（完整）

```
1 m = aml.Model("transport")
2 F = m.add_set("F", ["FA", "FB"])
3 C = m.add_set("C", ["C1", "C2", "C3"])
4 supply = m.add_param("s", dim=1)
5 demand = m.add_param("d", dim=1)
6 cost = m.add_param("c", dim=2)
7 supply.load({"FA":120.0, "FB":80.0})
8 demand.load({"C1":70.0, "C2":90.0,
9             "C3":40.0})
10 cost.load({"FA", "C1":4.0,
11           ("FA", "C2"):3.0,...})
12
13 x = m.add_var2("x", F, C,
14               aml.VarType.Continuous, lb=0.0)
15
16 m.minimize(aml.sum_over(F, lambda f:
17   aml.sum_over(C, lambda c:
18     cost[(f,c)] * x[(f,c)])))
19
```

```
1 m.add_constraints("sup", F,
2   lambda f:
3     aml.sum_over(C, lambda c:
4       x[(f,c)]*1.0) <= supply[f])
5
6 m.add_constraints("dem", C,
7   lambda c:
8     aml.sum_over(F, lambda f:
9       x[(f,c)]*1.0) >= demand[c])
10
11 opts = aml.SolveOptions()
12 r = m.solve(opts)
13 print(r.objective_value) # 570.0
14
```

完整示例： demo_03_aml_transport.py

SCUC 问题定义

安全约束机组组合 (SCUC)

Security-Constrained Unit Commitment

本质：日前调度问题，决定未来 $T = 24$ （或 96）个时段内：

- ① 每台机组何时开机/停机（整数决策）
- ② 每台机组出力多少 **MW**（连续决策）

优化目标：最小化总运行成本（含启动费、空载费、发电费）

约束体系：

- 功率平衡（需量 = 供给）
- 出力上下限（取决于开停状态）
- 爬坡速率（相邻时段出力变化）
- 最小开/停机时间
- 网络安全约束（N-1 准则）

决策变量集合

$u_{g,t} \in \{0, 1\}$	开停状态
$p_{g,t} \geq 0$	出力 (MW)
$su_{g,t} \in \{0, 1\}$	启动标志
$sd_{g,t} \in \{0, 1\}$	停机标志
$r_{g,t} \geq 0$	备用量 (MW)

计算复杂度

SCUC 是 **NP** 难问题
 G 台机组， T 时段：
二值变量数 = $3 \times G \times T$
连续变量数 = $G \times T$

目标函数（最小化总成本）：

$$\min \sum_g \sum_t \left[c_g^{nl} u_{g,t} + c_g^e p_{g,t} + c_g^{su} su_{g,t} \right]$$

其中 c^{nl} ：空载成本， c^e ：发电边际成本， c^{su} ：启动成本

功率平衡约束：

$$\sum_g p_{g,t} = D_t \quad \forall t$$

(D_t ： t 时段系统负荷)

出力界约束（依赖开停状态）：

$$\underline{P}_g u_{g,t} \leq p_{g,t} \leq \bar{P}_g u_{g,t} \quad \forall g, t$$

爬坡约束：

$$p_{g,t} - p_{g,t-1} \leq R_g \quad \forall g, t > 1$$

$$p_{g,t-1} - p_{g,t} \leq R_g \quad \forall g, t > 1$$

启停逻辑：

$$su_{g,t} - sd_{g,t} = u_{g,t} - u_{g,t-1} \quad \forall g, t$$

$$su_{g,t} + sd_{g,t} \leq 1$$

SCUC 约束：最小开/停机时间

最小开机时间 (T_g^{up}):

机组 g 在 t 期启动后，必须连续开机至少 T_g^{up} 个时段:

$$\sum_{\tau=t}^{\min(t+T_g^{up}-1, T)} u_{g,\tau} \geq T_g^{up} \cdot su_{g,t} \quad \forall t$$

最小停机时间 (T_g^{dn}):

机组 g 在 t 期停机后，必须连续停机至少 T_g^{dn} 个时段:

$$\sum_{\tau=t}^{\min(t+T_g^{dn}-1, T)} (1 - u_{g,\tau}) \geq T_g^{dn} \cdot sd_{g,t} \quad \forall t$$

分段报价线性化:

机组出力成本用分段线性函数近似:

$$c_g(p) = \sum_{k=1}^K c_{g,k} \delta_{g,k}$$

其中 $\delta_{g,k}$ 为各段出力量，满足:

$$\sum_k \delta_{g,k} = p_g, \quad 0 \leq \delta_{g,k} \leq \Delta \bar{P}_{g,k}$$

整体规模

典型日前调度 ($G = 100, T = 24$):

~ 7200 二值变量

~ 2400 连续变量

~ 20000 约束

求解时限: ≤ 10 分钟

AML 建模 SCUC: 数据与变量定义

```
1 gen_data = {  
2     "G1": dict(pmin=50, pmax=200,  
3               ramp=120, no_load=500,  
4               bid=30, startup=1000),  
5     "G2": dict(pmin=30, pmax=150,  
6               ramp=90, no_load=300,  
7               bid=45, startup=800),  
8 }  
9 load_mw = {"t1":150,"t2":200,  
10           "t3":260,"t4":240,  
11           "t5":180,"t6":120}  
12
```

```
1 m = aml.Model("mini_scuc")  
2 G = m.add_set("G", gen_names)  
3 T_set = m.add_ordered_set(  
4     "T", t_names) # t1..t6  
5
```

```
1 # 决策变量  
2 u = m.add_var2("u", G, T_set,  
3               aml.VarType.Binary)  
4 p = m.add_var2("p", G, T_set,  
5               aml.VarType.Continuous,  
6               lb=0.0)  
7 su = m.add_var2("su", G, T_set,  
8               aml.VarType.Binary)  
9 sd = m.add_var2("sd", G, T_set,  
10               aml.VarType.Binary)  
11
```

规模

$2 \times 6 = 12$ 连续变量

$3 \times 2 \times 6 = 36$ 二值变量

~ 60 个约束

AML 建模 SCUC: 目标与约束

目标函数:

```
1 m.minimize(aml.sum_over(G, lambda g:  
2     aml.sum_over(T_set, lambda t:  
3         d["no_load"] * u[(g,t)] +  
4         d["bid"] * p[(g,t)] +  
5         d["startup"] * su[(g,t)])))  
6
```

功率平衡约束:

```
1 for t in t_names:  
2     bal = aml.sum_over(G, lambda g:  
3         p[(g.values[0],t)] * 1.0)  
4     m.add_constraint(  
5         bal == load_mw[t],  
6         f"balance_{t}")  
7
```

出力上下界约束:

```
1 for g, d in gen_data.items():  
2     for t in t_names:  
3         # pmin*u <= p  
4         lhs = (1.0*p[(g,t)]  
5             - d["pmin"]*u[(g,t)])  
6         m.add_constraint(  
7             lhs >= 0.0, f"lb_{g}_{t}")  
8         # p <= pmax*u  
9         rhs = (d["pmax"]*u[(g,t)]  
10            - 1.0*p[(g,t)])  
11         m.add_constraint(  
12            rhs >= 0.0, f"ub_{g}_{t}")  
13
```

完整示例: demo_04_aml_scuc.py

AML 建模 SCUC: 结果解读

求解并提取结果:

```
1 opts = aml.SolveOptions()
2 opts.mip_gap_tol = 0.001
3 result = m.solve(opts)
4
5 print(result.termination_status)
6 # TerminationStatus.Optimal
7
8 print(f"总费用: ${result.objective_value:,.2f}")
9 # 总费用: $39,250.00
10
```

```
1 # 开停状态表
2 for g in gen_names:
3     row = [
4         f"{result.var_value(u[(g,t)):.0f}"
5         for t in t_names
6     ]
7     print(f"{g}: {row}")
8
9 # G1: ['1', '1', '1', '1', '1', '1']
10 # G2: ['0', '0', '1', '1', '1', '0']
11
```

典型输出:

	t1	t2	t3	t4	t5	t6
负荷 (MW)	150	200	260	240	180	120
G1 状态	1	1	1	1	1	1
G1 出力 (MW)	150.0	140.0	200.0	200.0	150.0	120.0
G2 状态	0	1	1	1	1	0
G2 出力 (MW)	0.0	60.0	60.0	40.0	30.0	0.0

JSON API: solve_json 接口

输入数据结构:

```
1 {
2   "config": {
3     "num_periods": 24,
4     "period_length_hr": 1.0,
5     "solver": "Auto",
6     "mip_gap": 0.001,
7     "solve_sced": true,
8     "solve_lmp": true
9   },
10  "generators": [
11    { "name": "G1", "bus": 0,
12      "pmin": 50.0, "pmax": 400.0,
13      "bid_segments": [
14        {"price": 20.0, "quantity": 150.0},
15        {"price": 35.0, "quantity": 150.0}
16      ], ... }
17  ],
18  "loads": [{"bus": 0, "p_mw": 300.0}],
19  "branches": [],
20  "profiles": {"load": [[0.8, ..., 0.95]]}
21 }
22
```

调用与解析:

```
1 import json, mipsolvers
2
3 json_in = json.dumps(input_data)
4 json_out = mipsolvers.scuc.solve_json(
5     json_in, indent=2)
6 res = json.loads(json_out)
7
8 scuc = res["scuc"]
9 print(scuc["converged"])      # True
10 print(scuc["cost"]["total"]) # $
11
```

输出结构:

```
1 {
2   "scuc": {
3     "converged": true,
4     "cost": {
5       "total": 85230.0,
6       "startup": 7000.0,
7       "energy": 78230.0
8     },
9     "dispatch": [[...], [...]],
10    "commitment": [[...], [...]],
11  }
12
```

JSON API: 结果分析示例

```
1 dispatch = scuc["dispatch"]
2 commitment = scuc["commitment"]
3 lmp = scuc["lmp"]
4 gen_names = ["G1", "G2", "G3"]
5
6 # 逐时段打印调度计划
7 print(f"{'t':>3} {'Load':>7}", end="")
8 for gn in gen_names:
9     print(f" {gn:>8}", end="")
10 print(f" {'LMP':>8}")
11
12 for t in range(24):
13     load_t = load_profile[t]*300.0
14     print(f"{'t'+1:3} {load_t:7.1f}", end="")
15     for gi in range(len(gen_names)):
16         print(f" {dispatch[gi][t]:8.1f}",
17               end="")
18     print(f" {lmp[t]:8.2f}")
19
```

边际电价 LMP 分析:

```
1 import matplotlib.pyplot as plt
2
3 hours = list(range(1, 25))
4 fig, (ax1, ax2) = plt.subplots(
5     2, 1, figsize=(8, 5))
6
7 ax1.plot(hours,
8         [load_profile[t]*300 for t in range(24)])
9 ax1.set_ylabel("Load (MW)")
10
11 ax2.plot(hours, lmp,
12         color='red', marker='o',
13         markersize=3)
14 ax2.set_ylabel("LMP ($/MWh)")
15 ax2.set_xlabel("Hour")
16
17 plt.tight_layout()
18 plt.savefig("lmp_curve.png")
19
```

完整示例: demo_05_scuc_api.py

已完成工作

核心框架：

- ✓ Engine 层：
LP/QP/NLP/MILP/MINLP/LE/NLE 统一接口
- ✓ AML 代数建模库（16 头文件 + 2 实现文件）
- ✓ HiGHS 集成（LP 及 MILP 开源求解器）
- ✓ SCIP 集成（大规模 MILP 开源求解器）
- ✓ 本地自研求解器（内点法、PDLP、LCQP、分支定界）

Python 接口：

- ✓ pybind11 绑定，Python 3.8 兼容
- ✓ `mipsolvers.engine` 底层接口
- ✓ `mipsolvers.aml` 代数建模接口
- ✓ `mipsolvers.scuc` SCUC 专用接口

文档：

- ✓ Python API 参考手册
(`docs/aml_python_api.md`)
- ✓ 5 个可运行 Python 示例
- ✓ 本教程幻灯片

近期工作计划

1. AC 最优潮流 (AC-OPF)

- 基于 NLP 接口实现完整 AC-OPF
- 支持 IEEE 测试系统 (14/30/118 节点)
- SDP 松弛近似 (凸化)

2. Benders 分解算法

- 支持大规模 SCUC 加速求解
- 主问题 (Unit Commitment) + 子问题 (SCED) 解耦
- 适用于 $G > 200$ 台机组的场景

3. 验证测试集

- IEEE RTS-96 标准测试系统
- 对比 Gurobi / HiGHS 求解质量与速度

4. 网络安全约束 (N-1)

- 将输电网络约束 (PTDF) 纳入 SCUC
- N-1 准则: 单一故障后仍满足约束

5. 分段报价完整支持

- SCUC 模块支持多段线性报价曲线
- 边际电价 (LMP) 精确计算
- 竞价区域划分

里程碑

目标: 在标准测试系统上达到与商业求解器相当的精度, 求解时间 < 5 分钟

中期规划

随机机组组合 (Stochastic UC)

- 考虑风电、光伏出力不确定性
- 基于情景树 (Scenario Tree)
- 两阶段随机规划框架：
 - 第一阶段：机组开停决策 (here-and-now)
 - 第二阶段：实时调度调整 (wait-and-see)

鲁棒机组组合 (Robust UC)

- 不确定性用集合描述 (椭球体、多面体)
- 列约束生成 (C&CG) 算法

多时间尺度协同

- 日前 (24h) $\times$ 日内滚动 (4h) $\times$ 实时 (15min)
- 储能 (电池) 的多时段套利建模
- 需求响应 (DR) 资源聚合

接口扩展

- Julia / MATLAB 绑定
- REST API 服务化部署
- 与 PowerModels.jl 对接

长期目标

打造电力系统运行优化全栈解决方案，覆盖从稳态分析到市场出清的完整链路

总结

MIPSolvers 提供了:

- ① 统一求解器接口—LP / MILP / NLP / MINLP / LE / NLE, 通过 `solver="Auto"` 自动调度 HiGHS / SCIP / 自研求解器
- ② 代数建模库 (AML) —以接近数学公式的方式建模, 支持多维索引变量和批量约束生成
- ③ SCUC 专用模块—JSON 输入/输出, 自动完成 UC + SCED + LMP 计算
- ④ Python 3.8 绑定—与 NumPy/Pandas/Matplotlib 无缝配合

配套示例文件:

- `demo_01_lp_direct.py`
生产计划 LP
- `demo_02_milp_knapsack.py`
0/1 背包 MILP
- `demo_03_aml_transport.py`
运输问题 (AML)
- `demo_04_aml_scuc.py`
迷你 SCUC (AML)
- `demo_05_scuc_api.py`
完整 SCUC (JSON API)

运行方式

```
python3.8 demo_XX.py
```

谢谢!

附录：完整安装与构建流程

```
1 # 1. 克隆仓库
2 git clone https://github.com/.../MIPSolvers.git
3 cd MIPSolvers
4
5 # 2. 构建 C++ 库与测试
6 cmake -B build_mipsolvers -DCMAKE_BUILD_TYPE=Release
7 cmake --build build_mipsolvers --parallel 8
8 cd build_mipsolvers && ctest --output-on-failure
9
10 # 3. 构建 Python 模块 (.so)
11 cmake -B build_py \
12     -DCMAKE_BUILD_TYPE=Release \
13     -DPython_EXECUTABLE=$(which python3.8)
14 cmake --build build_py --parallel 8
15
16 # 4. 测试 Python 模块
17 python3.8 -c "
18 import sys; sys.path.insert(0, 'build_py')
19 import mipsolvers
20 print(mipsolvers.engine.list_solvers('LP'))
21 print(mipsolvers.engine.list_solvers('MILP'))
22 "
23
```

附录：求解器选择指南

问题类型	规模	首选求解器	备选	说明
LP	小/中	HiGHS	NativeLCQP	开源，速度快
LP	大型	Gurobi	HiGHS	商业许可证
LP	流式/在线	NativePDLP	—	一阶方法，低内存
MILP	小/中	HiGHS	SCIP	开源首选
MILP	大型	Gurobi	SCIP	商业许可证
MILP	研究	SCIP	NativeBnC	强切平面支持
QP	凸	HiGHS	NativeLCQP	需 $Q \succeq 0$
NLP	稀疏	NativePMLP	—	内点法
MINLP	通用	NativeBnC	—	自研分支定界

注意事项：

- `solver="Auto"` 会按优先级（Gurobi > HiGHS > SCIP > Native...）自动选择
- Gurobi 需要有效许可证，否则 Auto 会跳过
- 对于 SCUC ($G < 100, T = 24$)，HiGHS 通常可在 1 分钟内求解